

Lab5 - Funciones y Disparadores SQL

**Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén
Sánchez, Inma Hernández, Margarita Cruz, Carlos Aré-
valo, David Ruiz**



Escuela Técnica Superior de
Ingeniería Informática

1. Objetivo	1
2. Preparación del entorno	2
3. Control de versiones	3
4. Funciones SQL	4
4.1. Características de las funciones	4
4.2. Sintaxis básica	4
4.3. Ejemplo 1: Función para calcular nota media de un estudiante	5
4.4. Ejemplo 2: Función para verificar si un estudiante está matriculado en una asignatura	6
4.5. Ejemplo 3: Función para verificar límite de grupos por asignatura	7
5. Funciones auxiliares para triggers	8
5.1. Función: f_student_group_limit	8
6. Disparadores (Triggers)	11
6.1. Características de los disparadores	11
6.2. Sintaxis básica	11
6.3. Valores NEW y OLD	12
6.4. Cuándo usar BEFORE vs AFTER	12
7. Triggers en GradesDB	13
7.1. Trigger RN08: Edad mínima para selectividad	18
7.2. Trigger RN02: Nota solo en grupos donde está matriculado	18
7.3. Trigger RN17: Una nota por asignatura, convocatoria y año	18
7.4. Trigger RN05: Límite de cambio en notas	19
7.5. Trigger RN03: Máximo de profesores por grupo	19
7.6. Triggers RN04 y RN07: Límites de grupos por estudiante	19
7.7. Triggers RN06: Límites de grupos por asignatura	20
8. Carga y verificación	21
9. Buenas prácticas con Triggers y Funciones	30
9.1. Separación de responsabilidades	30
9.2. Mensajes de error descriptivos	30
9.3. Nomenclatura consistente	30
9.4. Uso de funciones auxiliares	31
10. Ejercicios propuestos	32

Objetivo

El objetivo de esta práctica es implementar funciones y disparadores en SQL. El alumno aprenderá a:

- Crear funciones SQL que retornan valores y pueden usarse en consultas.
- Implementar disparadores (triggers) para validar reglas de negocio automáticamente.
- Usar disparadores BEFORE INSERT/UPDATE para validar datos antes de modificarlos.
- Combinar funciones auxiliares con disparadores para código reutilizable.

Los procedimientos almacenados ya se han visto en el L4. En este laboratorio nos centraremos en funciones (que sí retornan valores) y disparadores (que se ejecutan automáticamente ante eventos).

Preparación del entorno

Abre HeidiSQL y conéctate con el usuario `iissi_user` a la base de datos `GradesDB`. Asegúrate de haber ejecutado previamente los scripts `createDB.sql` y `populateDB.sql`.

Crea dos archivos: `functions.sql` para las funciones auxiliares y `triggers.sql` para los disparadores. Se ejecutarán en ese orden porque los triggers llaman a las funciones.

Control de versiones

Continuaremos trabajando con el repositorio `GradesDB` creado en L1.

Al inicio del laboratorio, añade el archivo y haz commit:

```
git add functions.sql triggers.sql
git commit -m "Añadidos functions.sql y triggers.sql para L5"
```

Al finalizar el laboratorio, haz push al repositorio remoto:

```
git add functions.sql triggers.sql
git commit -m "Completado L5: Funciones y disparadores SQL"
git push origin main
```

Funciones SQL

Las funciones son similares a los procedimientos pero tienen una diferencia fundamental: **las funciones devuelven un valor** mediante `RETURN` y pueden usarse directamente en consultas `SELECT`, `WHERE`, `ORDER BY`, etc.

4.1. Características de las funciones

- Deben declarar el tipo de dato que retornan mediante `RETURNS`.
- Usan `RETURN` para devolver el resultado.
- Pueden usarse en cualquier lugar donde se usaría una expresión o valor.
- Requieren especificar características como `DETERMINISTIC` o `READS SQL DATA`.
- No pueden modificar datos (no pueden usar `INSERT`, `UPDATE`, `DELETE`).

4.2. Sintaxis básica

```
DELIMITER //
```

```
CREATE OR REPLACE FUNCTION nombre_funcion(  
    parametro1 TIPO,  
    parametro2 TIPO  
) RETURNS TIPO_RETORNO  
DETERMINISTIC  
READS SQL DATA  
BEGIN
```

```

DECLARE v_resultado TIPO_RETORNO;

-- Lógica de la función
-- ...

RETURN v_resultado;
END //
DELIMITER ;

```

Características importantes:

- **DETERMINISTIC** : La función siempre devuelve el mismo resultado para los mismos parámetros.
- **READS SQL DATA** : Indica que la función lee datos pero no los modifica.
- Se debe usar **DELIMITER** igual que con los procedimientos.

4.3. Ejemplo 1: Función para calcular nota media de un estudiante

Esta función calcula el promedio de todas las notas de un estudiante.

f_student_average.sql

```

--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Función para calcular la nota media de un estudiante
--

USE GradesDB;

DELIMITER //
CREATE OR REPLACE FUNCTION f_student_average(
    p_student_id INT
) RETURNS DECIMAL(4,2)
BEGIN
    DECLARE v_average DECIMAL(4,2) DEFAULT 0;

    SELECT COALESCE(AVG(grade_value), 0) INTO v_average
    FROM grades
    WHERE student_id = p_student_id;

    RETURN v_average;
END //
DELIMITER ;

```

Para ejecutarla:

```
-- Usar en una consulta SELECT
SELECT f_student_average(6) AS 'Promedio del estudiante 6';

-- Usarla junto con otros datos
SELECT p.person_id, p.first_name, p.last_name,
       f_student_average(s.student_id) AS promedio
FROM people p
JOIN students s ON s.student_id = p.person_id
ORDER BY promedio DESC;

-- Usar en una cláusula WHERE
SELECT p.first_name, p.last_name
FROM people p
JOIN students s ON s.student_id = p.person_id
WHERE f_student_average(s.student_id) >= 7.0;
```

Observe lo siguiente:

- La función retorna `DECIMAL(4,2)` especificado en `RETURNS`.
- Se inicializa `v_average` con `DEFAULT 0` para devolver 0 si el estudiante no tiene notas (en lugar de NULL).
- La función se invoca directamente en consultas, sin necesidad de `CALL`.
- Puede usarse en `SELECT`, `WHERE`, `ORDER BY`, etc.
- Es más concisa que un procedimiento con parámetro OUT para este caso.

4.4. Ejemplo 2: Función para verificar si un estudiante está matriculado en una asignatura

Esta es una función booleana (devuelve TRUE/FALSE) útil para validaciones. Su definición se incorpora al archivo `functions.sql`, mostrado al final de esta sección.

Para ejecutarla:

```
-- Listar asignaturas con indicador de matrícula del estudiante 6 del grado 3
SELECT s.subject_id, s.subject_name,
       f_is_student_enrolled(6, s.subject_id) AS matriculado
FROM subjects s
WHERE s.degree_id = 3;

-- Usar en un WHERE
SELECT s.subject_name
FROM subjects s
WHERE f_is_student_enrolled(6, s.subject_id) = TRUE;
```

Observe lo siguiente:

- Devuelve un valor `BOOLEAN` (`TRUE` o `FALSE`).
- La expresión `v_exists > 0` devuelve un booleano directamente.
- Las funciones booleanas son muy útiles para validaciones y filtros.
- Pueden usarse en cláusulas `WHERE` con comparaciones `= TRUE` o `= FALSE`.

4.5. Ejemplo 3: Función para verificar límite de grupos por asignatura

Esta función verifica si una asignatura ha alcanzado el número máximo de grupos permitidos para un tipo de actividad (1 para Teoría, 2 para Laboratorio) en un año académico específico. Su definición también forma parte de `functions.sql`.

Para ejecutarla:

```
-- Verificar si la asignatura 1 ya tiene el máximo de grupos de teoría en 2025
SELECT f_subject_group_limit_reached(1, 'Teoría', 2025) AS 'Límite alcanzado';

-- Listar asignaturas que han alcanzado el límite de grupos de teoría
SELECT s.subject_id, s.subject_name, s.acronym
FROM subjects s
WHERE f_subject_group_limit_reached(s.subject_id, 'Teoría', 2025) = TRUE;
```

Observe lo siguiente:

- Usa lógica condicional `IF ... THEN ... ELSEIF ... END IF` dentro de la función.
- Implementa lógica de negocio: diferentes límites para teoría (1) y laboratorio (2).
- Retorna un booleano indicando si se alcanzó el límite máximo.
- La función recibe únicamente los datos que identifican el conjunto de grupos que debe contar.

Funciones auxiliares para triggers

Antes de implementar los triggers, necesitamos definir una función auxiliar adicional que encapsula lógica de validación reutilizable. Esta función complementa las ya vistas en los ejemplos anteriores.

5.1. Función: `f_student_group_limit`

Verifica si un estudiante ha alcanzado el límite de grupos permitidos para una actividad en una asignatura (1 grupo de teoría o 1 grupo de laboratorio por asignatura).

Las tres funciones auxiliares se reúnen en un único script, que debe ejecutarse antes de crear los triggers:

functions.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Septiembre 2026  
-- Descripción: Funciones auxiliares para las reglas de negocio de GradesDB  
--  
  
USE GradesDB;  
  
DELIMITER //  
  
CREATE OR REPLACE FUNCTION f_student_group_limit(  

```

```

    p_student_id INT,
    p_subject_id INT,
    p_activity VARCHAR(15)
)
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count INT;
    DECLARE v_result BOOLEAN DEFAULT FALSE;
    SELECT COUNT(*) INTO v_count
    FROM group_enrollments ge
    JOIN groups g ON g.group_id = ge.group_id
    WHERE ge.student_id = p_student_id
        AND g.subject_id = p_subject_id
        AND g.activity = p_activity;

    IF p_activity = 'Teoría' THEN
        SET v_result = (v_count >= 1);
    ELSEIF p_activity = 'Laboratorio' THEN
        SET v_result = (v_count >= 1);
    END IF;
    RETURN v_result;
END//

CREATE OR REPLACE FUNCTION f_is_student_enrolled(
    p_student_id INT,
    p_subject_id INT
)
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_exists INT;
    SELECT COUNT(*) INTO v_exists
    FROM subject_enrollments
    WHERE student_id = p_student_id
        AND subject_id = p_subject_id;
    RETURN v_exists > 0;
END//

CREATE OR REPLACE FUNCTION f_subject_group_limit_reached(
    p_subject_id INT,
    p_activity VARCHAR(15),
    p_academic_year YEAR
)
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN

```

```

DECLARE v_count INT;
DECLARE v_result BOOLEAN DEFAULT FALSE;

SELECT COUNT(*) INTO v_count
FROM groups
WHERE subject_id = p_subject_id
      AND activity = p_activity
      AND academic_year = p_academic_year;

IF p_activity = 'Teoría' THEN
    SET v_result = (v_count >= 1);
ELSEIF p_activity = 'Laboratorio' THEN
    SET v_result = (v_count >= 2);
END IF;

RETURN v_result;
END//

DELIMITER ;

```

Observe lo siguiente:

- Cuenta los grupos en los que el estudiante está matriculado para una asignatura y actividad específicas.
- Retorna TRUE si el estudiante ya alcanzó el límite para esa actividad.
- Se usa en los triggers RN04 para validar límites de grupos por estudiante.

Resumen de funciones auxiliares para triggers:

- `f_is_student_enrolled` : Verifica matrícula en asignatura (vista en Ejemplo 2)
- `f_subject_group_limit_reached` : Verifica límites de grupos por asignatura (vista en Ejemplo 3)
- `f_student_group_limit` : Verifica límites de grupos por estudiante (recién definida)

Disparadores (Triggers)

Los disparadores son bloques de código SQL que se ejecutan **automáticamente** cuando ocurre un evento específico en una tabla (INSERT, UPDATE o DELETE). Son fundamentales para implementar reglas de negocio y validaciones complejas.

6.1. Características de los disparadores

- Se ejecutan automáticamente, sin necesidad de llamarlos explícitamente.
- Pueden ejecutarse BEFORE (antes) o AFTER (después) del evento.
- Tienen acceso a los valores NEW (nuevos) y OLD (antiguos) de las filas afectadas.
- Pueden lanzar errores con `SIGNAL` para cancelar la operación.
- Se definen por cada operación (INSERT, UPDATE, DELETE) y momento (BEFORE, AFTER).

6.2. Sintaxis básica

```
DELIMITER //
```

```
CREATE OR REPLACE TRIGGER nombre_trigger
```

```
{BEFORE | AFTER} {INSERT | UPDATE | DELETE | INSERT OR UPDATE} ON
```

```
nombre_tabla
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    -- Código del trigger
```

```
    -- Puede usar NEW.columna (valor nuevo)
```

```
-- Puede usar OLD.columna (valor antiguo en UPDATE/DELETE)
END //
DELIMITER ;
```

Nota importante: MariaDB permite combinar operaciones con `OR`, por ejemplo `BEFORE INSERT OR UPDATE`, lo que evita tener que crear triggers separados para cada operación cuando la lógica es la misma.

6.3. Valores NEW y OLD

Operación	NEW	OLD
INSERT	Valores siendo insertados	No disponible
UPDATE	Valores nuevos después del UPDATE	Valores antiguos antes del UPDATE
DELETE	No disponible	Valores siendo borrados

6.4. Cuándo usar BEFORE vs AFTER

- **BEFORE:** Para validar datos y potencialmente cancelar la operación con `SIGNAL`.
- **AFTER:** Para realizar acciones después de que la operación se completó exitosamente.

En GradesDB, **todos los triggers son BEFORE** porque se usan para validar reglas de negocio.

Triggers en GradesDB

Añade a `triggers.sql` los disparadores que validan las reglas de negocio de GradesDB. El script completo es el siguiente y debe ejecutarse después de `functions.sql`.

triggers.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Triggers para las reglas de negocio de GradesDB
--

USE GradesDB;

DELIMITER //

CREATE OR REPLACE TRIGGER t_biu_students_rn08
BEFORE INSERT OR UPDATE ON students
FOR EACH ROW
BEGIN
    DECLARE v_age TINYINT;
    IF NEW.access_method = 'Selectividad' THEN
        SELECT age INTO v_age FROM people WHERE person_id = NEW.student_id;
        IF v_age < 16 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'RN08: No se puede acceder por
Selectividad con menos de 16 años';
        END IF;
    END IF;
END IF;
```

```
END//
```

```
CREATE OR REPLACE TRIGGER t_biu_grades_rn02
```

```
BEFORE INSERT OR UPDATE ON grades
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE v_count INT;
```

```
    SELECT COUNT(*) INTO v_count
```

```
    FROM group_enrollments
```

```
    WHERE student_id = NEW.student_id
```

```
        AND group_id = NEW.group_id;
```

```
    IF v_count = 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'RN02: El alumno no pertenece al grupo  
indicado';
```

```
    END IF;
```

```
END//
```

```
CREATE OR REPLACE TRIGGER t_biu_grades_rn17
```

```
BEFORE INSERT OR UPDATE ON grades
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE v_subject_id INT;
```

```
    DECLARE v_academic_year YEAR;
```

```
    DECLARE v_exists INT;
```

```
    SELECT subject_id, academic_year INTO v_subject_id, v_academic_year
```

```
    FROM groups
```

```
    WHERE group_id = NEW.group_id;
```

```
    SELECT COUNT(*) INTO v_exists
```

```
    FROM grades g
```

```
    JOIN groups gr ON gr.group_id = g.group_id
```

```
    WHERE g.student_id = NEW.student_id
```

```
        AND g.exam_call = NEW.exam_call
```

```
        AND gr.subject_id = v_subject_id
```

```
        AND gr.academic_year = v_academic_year
```

```
        AND (NEW.grade_id IS NULL OR g.grade_id <> NEW.grade_id);
```

```
    IF v_exists > 0 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'RN17: Ya existe una nota para la misma  
asignatura, convocatoria y año académico';
```

```
    END IF;
```

```
END//
```

```
CREATE OR REPLACE TRIGGER t_bu_grades_rn05
```

```
BEFORE UPDATE ON grades
```

```
FOR EACH ROW
```

```

BEGIN
  IF ABS(NEW.grade_value - OLD.grade_value) > 4 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'RN05: No se puede modificar la nota en más
de 4 puntos';
  END IF;
END//

CREATE OR REPLACE TRIGGER t_bi_teaching_loads_rn03
BEFORE INSERT ON teaching_loads
FOR EACH ROW
BEGIN
  DECLARE v_count INT;

  SELECT COUNT(*) INTO v_count
  FROM teaching_loads
  WHERE group_id = NEW.group_id;

  IF v_count >= 2 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'RN03: El grupo ya tiene 2 profesores
asignados';
  END IF;
END//

CREATE OR REPLACE TRIGGER t_bu_teaching_loads_rn03
BEFORE UPDATE ON teaching_loads
FOR EACH ROW
BEGIN
  DECLARE v_count INT;

  SELECT COUNT(*) INTO v_count
  FROM teaching_loads
  WHERE group_id = NEW.group_id
    AND NOT (professor_id = OLD.professor_id AND group_id =
OLD.group_id);

  IF v_count >= 2 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'RN03: El grupo ya tiene 2 profesores
asignados';
  END IF;
END//

CREATE OR REPLACE TRIGGER t_bi_group_enrollments_rn04
BEFORE INSERT ON group_enrollments
FOR EACH ROW
BEGIN
  DECLARE v_subject_id INT;
  DECLARE v_activity VARCHAR(15);

```

```

SELECT subject_id, activity INTO v_subject_id, v_activity
FROM groups
WHERE group_id = NEW.group_id;

IF NOT f_is_student_enrolled(NEW.student_id, v_subject_id) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RN07: El alumno debe estar matriculado en
la asignatura';
END IF;

IF f_student_group_limit(NEW.student_id, v_subject_id, v_activity) THEN
    IF v_activity = 'Teoría' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN04: Solo puede haber un grupo de
teoría por asignatura y alumno';
    ELSE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN04: Solo puede haber dos grupos de
laboratorio por asignatura y alumno';
    END IF;
END IF;
END//

CREATE OR REPLACE TRIGGER t_bu_group_enrollments_rn04
BEFORE UPDATE ON group_enrollments
FOR EACH ROW
BEGIN
    DECLARE v_subject_id INT;
    DECLARE v_activity VARCHAR(15);
    DECLARE v_old_subject_id INT;
    DECLARE v_old_activity VARCHAR(15);
    SELECT subject_id, activity INTO v_subject_id, v_activity
    FROM groups
    WHERE group_id = NEW.group_id;

    SELECT subject_id, activity INTO v_old_subject_id, v_old_activity
    FROM groups
    WHERE group_id = OLD.group_id;

    IF NOT f_is_student_enrolled(NEW.student_id, v_subject_id) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN07: El alumno debe estar matriculado en
la asignatura';
    END IF;

    IF (NEW.student_id <> OLD.student_id
        OR v_subject_id <> v_old_subject_id
        OR v_activity <> v_old_activity)
        AND f_student_group_limit(NEW.student_id, v_subject_id, v_activity)
THEN

```

```

        IF v_activity = 'Teoría' THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'RN04: Solo puede haber un grupo de
teoría por asignatura y alumno';
        ELSE
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'RN04: Solo puede haber dos grupos de
laboratorio por asignatura y alumno';
        END IF;
    END IF;
END//

CREATE OR REPLACE TRIGGER t_bi_groups_rn06
BEFORE INSERT ON groups
FOR EACH ROW
BEGIN
    IF f_subject_group_limit_reached(NEW.subject_id, NEW.activity,
NEW.academic_year) THEN
        IF NEW.activity = 'Teoría' THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'RN06: Solo puede existir un grupo de
teoría por asignatura y año académico';
        ELSE
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'RN06: Solo pueden existir dos grupos de
laboratorio por asignatura y año académico';
        END IF;
    END IF;
END//

CREATE OR REPLACE TRIGGER t_bu_groups_rn06
BEFORE UPDATE ON groups
FOR EACH ROW
BEGIN
    IF NEW.subject_id <> OLD.subject_id
    OR NEW.activity <> OLD.activity
    OR NEW.academic_year <> OLD.academic_year THEN
        IF f_subject_group_limit_reached(NEW.subject_id, NEW.activity,
NEW.academic_year) THEN
            IF NEW.activity = 'Teoría' THEN
                SIGNAL SQLSTATE '45000'
                    SET MESSAGE_TEXT = 'RN06: Solo puede existir un grupo
de teoría por asignatura y año académico';
            ELSE
                SIGNAL SQLSTATE '45000'
                    SET MESSAGE_TEXT = 'RN06: Solo pueden existir dos
grupos de laboratorio por asignatura y año académico';
            END IF;
        END IF;
    END IF;
END IF;
END//

```

```
END//
```

```
DELIMITER ;
```

7.1. Trigger RN08: Edad mínima para selectividad

Regla de negocio: Un alumno que accede por selectividad debe tener al menos 16 años.

Observe lo siguiente:

- `BEFORE INSERT OR UPDATE` : Se ejecuta antes de insertar o actualizar en la tabla `students`.
- `FOR EACH ROW` : Se ejecuta por cada fila afectada.
- Se declara una variable local `v_age` para consultar la edad desde la tabla `people`.
- `NEW.access_method` : Accede al valor del método de acceso que se está insertando/actualizando.
- `SIGNAL SQLSTATE '45000'` : Lanza un error personalizado que cancela la operación.
- El error incluye un mensaje descriptivo con el prefijo de la regla (RN08).

7.2. Trigger RN02: Nota solo en grupos donde está matriculado

Regla de negocio: Un alumno solo puede tener notas en grupos a los que pertenece.

Observe lo siguiente:

- Verifica la existencia del estudiante en el grupo mediante un `COUNT`.
- Si no existe matrícula en el grupo (`v_count = 0`), lanza un error.
- Previene inconsistencias: no pueden existir notas de estudiantes que no están en el grupo.

7.3. Trigger RN17: Una nota por asignatura, convocatoria y año

Regla de negocio: Un alumno no puede tener más de una nota para la misma asignatura, convocatoria y año académico.

Observe lo siguiente:

- Primero obtiene la asignatura y año académico del grupo.
- Busca si ya existe otra nota con las mismas características.
- La condición `NEW.grade_id IS NULL OR g.grade_id <> NEW.grade_id` diferencia entre `INSERT (NULL)` y `UPDATE` (excluye la fila actual).

- Requiere un JOIN para relacionar notas con grupos y obtener la asignatura.

7.4. Trigger RN05: Límite de cambio en notas

Regla de negocio: Una nota no puede modificarse en más de 4 puntos.

Observe lo siguiente:

- Este trigger es solo `BEFORE UPDATE` (no aplica a `INSERT`).
- Usa `OLD.grade_value` para el valor antiguo y `NEW.grade_value` para el nuevo.
- `ABS()` calcula el valor absoluto para manejar tanto subidas como bajadas.
- Validación simple pero efectiva para prevenir cambios drásticos en notas.

7.5. Trigger RN03: Máximo de profesores por grupo

Regla de negocio: Un grupo no puede tener más de 2 profesores asignados.

Observe lo siguiente:

- Se definen **dos triggers separados**: uno para `INSERT` y otro para `UPDATE`, ya que la lógica de validación es ligeramente diferente.
- Aunque MariaDB permite `INSERT OR UPDATE`, en este caso se separan porque el trigger de `UPDATE` necesita excluir la fila actual del conteo.
- El trigger de `UPDATE` excluye la fila actual con `NOT (professor_id = OLD.professor_id AND group_id = OLD.group_id)`.
- Cuenta profesores existentes antes de permitir la asignación.
- Implementa el límite de 2 profesores por grupo.

7.6. Triggers RN04 y RN07: Límites de grupos por estudiante

Reglas de negocio:

- RN04: Un alumno solo puede estar en 1 grupo de teoría y 1 de laboratorio por asignatura.
- RN07: Un alumno debe estar matriculado en la asignatura antes de unirse a un grupo.

Estos triggers **usan funciones auxiliares** para código más limpio:

Observe lo siguiente:

- El trigger **llama a funciones** (`f_is_student_enrolled` y `f_student_group_limit`) en lugar de repetir la lógica.
- Esto hace el código más mantenible y reutilizable.
- Valida primero la matrícula en la asignatura (RN07).

- Luego valida el límite de grupos según el tipo de actividad (RN04).
- Las funciones se definen previamente en `functions.sql`.

7.7. Triggers RN06: Límites de grupos por asignatura

Regla de negocio: Solo puede haber 1 grupo de teoría y 2 de laboratorio por asignatura y año académico.

Observe lo siguiente:

- Usa la función `f_subject_group_limit_reached` para verificar el límite.
- El trigger de INSERT comprueba siempre si la combinación ha alcanzado su límite.
- El trigger de UPDATE solo comprueba el límite cuando cambian la asignatura, la actividad o el año académico; cambiar otros atributos no altera el conjunto contado.
- Diferentes mensajes de error según el tipo de actividad.
- La función encapsula la lógica compleja de conteo y validación.

Carga y verificación

Guarda las funciones y triggers anteriores en sus respectivos archivos. Si ya existen objetos con el mismo nombre, `CREATE OR REPLACE` los actualizará.

```
SOURCE functions.sql;
SOURCE triggers.sql;
```

Una vez instaladas las reglas procedurales, ejecuta la batería completa. `tests.sql` incorpora a los tests declarativos de L4 los casos correspondientes a RN003-RN008 y RN017.

tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Tests negativos para la BD de Grados
--
USE GradesDB;

-- =====
-- TABLA DE RESULTADOS
-- =====

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```

);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS','FAIL','ERROR')
)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //
DELIMITER ;

-- =====
-- TESTS (RN001 - RN016)
-- =====

-- Test RN001: Matrícula de honor requiere nota >= 9
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn001_mh_requirement()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN001', 'RN001: La MH requiere nota >= 9',
'PASS');

    CALL p_populate();

    UPDATE grades SET with_honors = 1 WHERE grade_id = 21;

    CALL p_log_test('RN001', 'ERROR: Se permitió MH con nota inferior a 9',
'FAIL');
END //
DELIMITER ;

-- Test RN017: No se permiten notas duplicadas por asignatura/convocatoria
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn017_duplicate_grade()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN017', 'RN017: No se permiten notas duplicadas
por asignatura/convocatoria', 'PASS');

    CALL p_populate();

    INSERT INTO grades (grade_id, student_id, group_id, grade_value,

```

```

exam_call, with_honors)
    VALUES (101, 6, 1, 6.0, 'Primera', 0);

    CALL p_log_test('RN017', 'ERROR: Se permitió duplicar nota en la misma
convocatoria', 'FAIL');
END //
DELIMITER ;

-- Test RN003: Un grupo no puede tener más de 2 profesores
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn003_professors_per_group()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN003', 'RN003: No se permite añadir un tercer
profesor al grupo', 'PASS');

    CALL p_populate();

    INSERT INTO teaching_loads (professor_id, group_id, credits) VALUES (5,
1, 1.0);

    CALL p_log_test('RN003', 'ERROR: Se asignó un tercer profesor al
grupo', 'FAIL');
END //
DELIMITER ;

-- Test RN004: Un alumno no puede pertenecer a más de un grupo de teoría y
uno de laboratorio por asignatura
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn004_student_group_limit()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN004', 'RN004: Un alumno no puede pertenecer a
más grupos de los permitidos', 'PASS');

    CALL p_populate();

    INSERT INTO group_enrollments (student_id, group_id) VALUES (6, 3);

    CALL p_log_test('RN004', 'ERROR: Se permitió un alumno en más grupos de
los permitidos', 'FAIL');
END //
DELIMITER ;

-- Test RN005: Una nota no puede alterarse en más de 4 puntos
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn005_grade_delta()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN005', 'RN005: No se puede modificar la nota en

```

```

más de 4 puntos', 'PASS');

CALL p_populate();

UPDATE grades SET grade_value = 0.5 WHERE grade_id = 1;

CALL p_log_test('RN005', 'ERROR: Se permitió modificar la nota en más
de 4 puntos', 'FAIL');
END //
DELIMITER ;

-- Test RN006: No puede haber más de un grupo de teoría y dos de
laboratorio por asignatura
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn006_extra_group()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN006', 'RN006: No se permite crear un segundo
grupo de teoría para la asignatura', 'PASS');

    CALL p_populate();

    INSERT INTO groups (group_id, subject_id, group_name, activity,
academic_year)
        VALUES (11, 11, 'T2', 'Teoría', 2024);

    CALL p_log_test('RN006', 'ERROR: Se creó un segundo grupo de teoría
para la misma asignatura', 'FAIL');
END //
DELIMITER ;

-- Test RN007: Un alumno sólo puede pertenecer a grupos de asignaturas en
las que está matriculado
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn007_subject_enrollment()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN007', 'RN007: No se puede añadir a un grupo sin
matrícula en la asignatura', 'PASS');

    CALL p_populate();

    INSERT INTO people (person_id, dni, first_name, last_name, age, email)
        VALUES (102, '20000002B', 'Test', 'Alumno', 20,
'nuevo@alum.us.es');
    INSERT INTO students (student_id, access_method) VALUES (102,
'Selectividad');
    INSERT INTO group_enrollments (student_id, group_id) VALUES (102, 1);

    CALL p_log_test('RN007', 'ERROR: Se permitió unir a un grupo sin

```

```

matrícula en la asignatura', 'FAIL');
END //
DELIMITER ;

-- Test RN008: Un alumno no puede acceder por selectividad con menos de 16
años
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn008_min_age()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN008', 'RN008: No se permite Selectividad con
menos de 16 años', 'PASS');

    CALL p_populate();

    INSERT INTO people (person_id, dni, first_name, last_name, age, email)
        VALUES (104, '20000004D', 'Test', 'Menor', 15, 'menor@alum.us.es');
    INSERT INTO students (student_id, access_method) VALUES (104,
'Selectividad');

    CALL p_log_test('RN008', 'ERROR: Se permitió Selectividad para un
menor', 'FAIL');
END //
DELIMITER ;

-- Test RN009: Los atributos obligatorios no pueden quedar a NULL
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn009_not_null_attributes()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN009', 'RN009: Los atributos obligatorios no
pueden quedar a NULL', 'PASS');

    CALL p_populate();

    INSERT INTO people (person_id, dni, first_name, last_name, age, email)
        VALUES (103, '20000003C', NULL, 'Campos', 22, 'null@alum.us.es');

    CALL p_log_test('RN009', 'ERROR: Se permitió dejar atributos
obligatorios a NULL', 'FAIL');
END //
DELIMITER ;

-- Test RN010: Los créditos de una asignatura pueden ser 6 o 12
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn010_subject_credits()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN010', 'RN010: Los créditos de una asignatura
pueden ser 6 o 12', 'PASS');

```

```

CALL p_populate();

INSERT INTO subjects (subject_id, degree_id, subject_name, acronym,
credits, course, subject_type)
VALUES (31, 3, 'Asignatura Créditos Inválidos', 'ACI', 8, 2,
'Obligatoria');

CALL p_log_test('RN010', 'ERROR: Se permitió una asignatura con
créditos inválidos', 'FAIL');
END //
DELIMITER ;

-- Test RN011: El valor de la nota está comprendido entre 0 y 10
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn011_grade_range()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN011', 'RN011: La nota debe estar entre 0 y 10',
'PASS');

    CALL p_populate();

    INSERT INTO grades (grade_id, student_id, group_id, grade_value,
exam_call, with_honors)
VALUES (201, 6, 1, 11.0, 'Primera', 0);

    CALL p_log_test('RN011', 'ERROR: Se permitió una nota fuera de rango',
'FAIL');
END //
DELIMITER ;

-- Test RN012: La edad de las personas está entre 16 y 70 años
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn012_people_age()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN012', 'RN012: La edad de las personas debe estar
entre 16 y 70', 'PASS');

    CALL p_populate();

    INSERT INTO people (person_id, dni, first_name, last_name, age, email)
VALUES (105, '20000005E', 'Edad', 'Fuera', 80, 'edad@us.es');

    CALL p_log_test('RN012', 'ERROR: Se permitió una edad fuera de rango',
'FAIL');
END //
DELIMITER ;

```

```
-- Test RN013: Los años de un grado están entre 3 y 6
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn013_degree_years()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN013', 'RN013: Los grados deben tener entre 3 y 6
años', 'PASS');

    CALL p_populate();

    INSERT INTO degrees (degree_id, degree_name, duration_years)
        VALUES (10, 'Grado Experimental', 2);

    CALL p_log_test('RN013', 'ERROR: Se permitió un grado con años fuera de
rango', 'FAIL');
END //
DELIMITER ;

-- Test RN014: El DNI está formado por 8 números y una letra
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn014_dni_format()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN014', 'RN014: El DNI debe tener 8 dígitos y una
letra', 'PASS');

    CALL p_populate();

    INSERT INTO people (person_id, dni, first_name, last_name, age, email)
        VALUES (106, 'INVALIDO', 'DNI', 'Incorrecto', 30, 'dni@us.es');

    CALL p_log_test('RN014', 'ERROR: Se permitió un DNI con formato
inválido', 'FAIL');
END //
DELIMITER ;

-- Test RN015: El año académico está entre 2000 y 2100
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn015_academic_year()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN015', 'RN015: El año académico debe estar entre
2000 y 2100', 'PASS');

    CALL p_populate();

    INSERT INTO groups (group_id, subject_id, group_name, activity,
academic_year)
        VALUES (20, 12, 'MD-T2025', 'Teoría', 1999);
```

```

    CALL p_log_test('RN015', 'ERROR: Se permitió un año académico fuera de
rango', 'FAIL');
END //
DELIMITER ;

-- Test RN016: El curso de una asignatura está entre 1 y 6
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn016_course_range()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN016', 'RN016: El curso de una asignatura debe
estar entre 1 y 6', 'PASS');

    CALL p_populate();

    INSERT INTO subjects (subject_id, degree_id, subject_name, acronym,
credits, course, subject_type)
        VALUES (30, 3, 'Asignatura Fuera de Curso', 'AFC', 6, 0,
'Obligatoria');

    CALL p_log_test('RN016', 'ERROR: Se permitió un curso fuera de rango',
'FAIL');
END //
DELIMITER ;

-- =====
-- ORQUESTADOR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_grados_tests()
BEGIN
    DELETE FROM test_results;

    CALL p_test_rn001_mh_requirement();
    CALL p_test_rn017_duplicate_grade();
    CALL p_test_rn003_professors_per_group();
    CALL p_test_rn004_student_group_limit();
    CALL p_test_rn005_grade_delta();
    CALL p_test_rn006_extra_group();
    CALL p_test_rn007_subject_enrollment();
    CALL p_test_rn008_min_age();
    CALL p_test_rn009_not_null_attributes();
    CALL p_test_rn010_subject_credits();
    CALL p_test_rn011_grade_range();
    CALL p_test_rn012_people_age();
    CALL p_test_rn013_degree_years();
    CALL p_test_rn014_dni_format();
    CALL p_test_rn015_academic_year();
    CALL p_test_rn016_course_range();

```

8. Carga y verificación

```
SELECT * FROM test_results ORDER BY execution_time, test_id;
SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;

CALL p_run_grados_tests();
```

```
SOURCE tests.sql;
```

El resultado esperado es:

Estado	Total
PASS	16

Buenas prácticas con Triggers y Funciones

9.1. Separación de responsabilidades

- **Funciones:** Encapsulan lógica de validación reutilizable.
- **Triggers:** Coordinan las validaciones y lanzan errores.

9.2. Mensajes de error descriptivos

- Incluir el código de la regla de negocio (ej: RN01, RN02).
- Explicar claramente qué violación se detectó.
- Facilitar el debugging y comprensión de problemas.

9.3. Nomenclatura consistente

- Triggers: `t_{bi|bu|bd}_{tabla}_{rn}`
 - `bi` = BEFORE INSERT
 - `bu` = BEFORE UPDATE
 - `bd` = BEFORE DELETE
 - `biu` = BEFORE INSERT OR UPDATE

- Funciones: `f_{descripcion}`
- Variables: `v_{nombre}`
- Parámetros: `p_{nombre}`

9.4. Uso de funciones auxiliares

Cuando la misma lógica se necesita en múltiples lugares:

1. Crear una función que encapsule la lógica.
2. Llamar a la función desde los triggers.
3. Mantener el código DRY (Don't Repeat Yourself).

Ejercicios propuestos

Implementa las siguientes funciones en `functions.sql` y los triggers en `triggers.sql`:

1. **Función:** `f_count_student_grades(p_student_id INT, p_exam_call VARCHAR(20))` que cuente cuántas notas tiene un estudiante en una convocatoria específica.
2. **Función:** `f_professor_total_credits(p_professor_id INT)` que calcule el total de créditos que imparte un profesor.
3. **Trigger:** Validar que un grado no pueda tener más de 240 créditos en total sumando todas sus asignaturas.
4. **Trigger:** Validar que un estudiante no pueda estar matriculado en más de 60 créditos por año académico.

Info

Versión PDF disponible